

Processo de Software

O que é um processo?

- .Fazer um bolo
- .Programar
- .Dar direções
- .Abrir uma conta
- O que estas coisas têm em comum?

Elementos do Processo

- .**Coloque** em uma **panela funda** o leite condensado, a margarina e o chocolate em pó.
- .**Cozinhe** no **fogão** em fogo médio e **mexa** sem parar com uma **colher de pau**.
- .**Cozinhe** até que o brigadeiro comece a desgrudar da panela.
- .**Deixe esfriar** bem, então **unte** as **mãos** com margarina, **faça as bolinhas** e **envolva**-as em chocolate granulado

Ferramentas

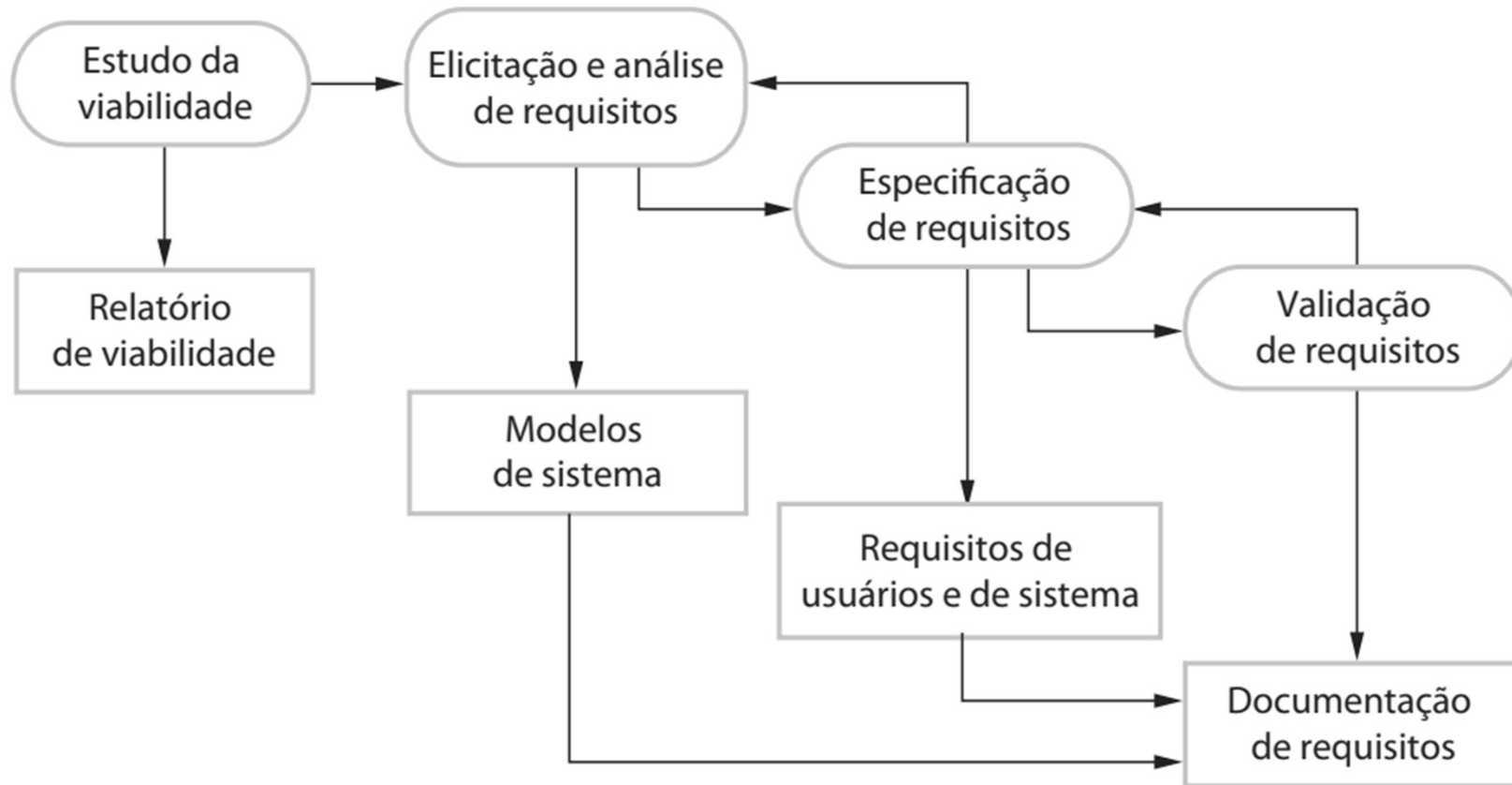
Atividades

Processo

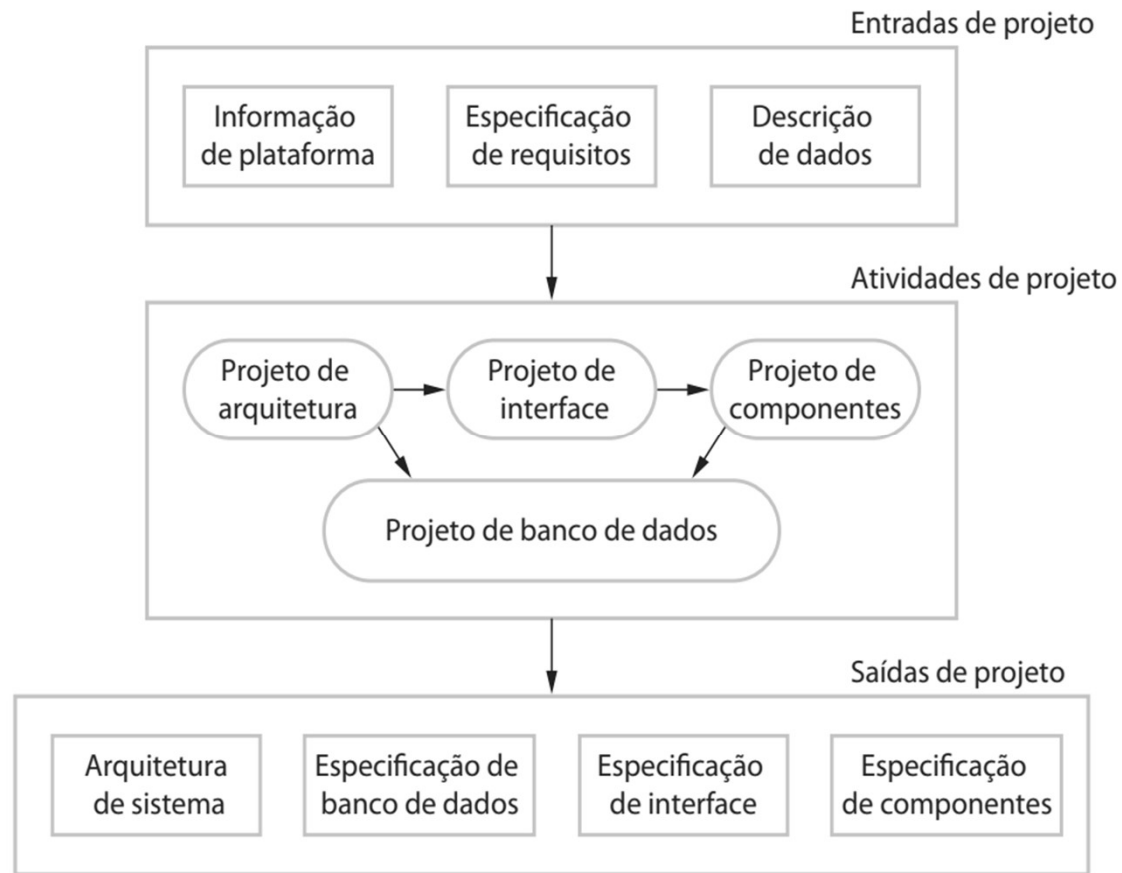
Processo de Software

- Um conjunto de atividades que guiam o desenvolvimento de um sistema de software
- Existem diversos processos, mas todos envolvem:
 - Especificação: descreve “o que” o sistema faz
 - Projeto: descreve “como” o sistema faz
 - Implementação: torna o sistema real
 - Validação: verifica se faz o que deve de forma satisfatória
 - Evolução: alterações para novas necessidades

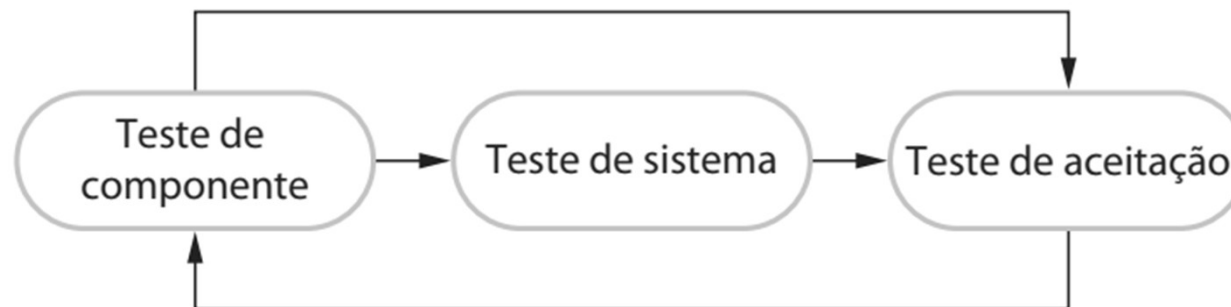
Especificação



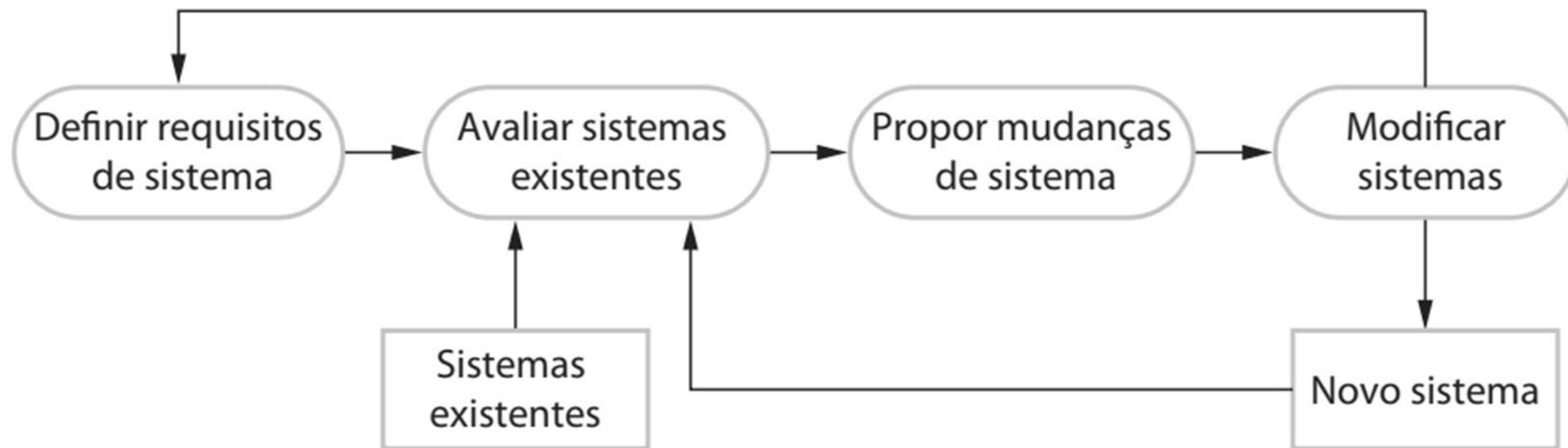
Projeto



Validação



Evolução



Descrição do Processo

- .Geralmente descrevemos através de atividades
 - e.g.: especificar o modelo de dados, projetar a interface
- .Também podemos incluir
 - Produtos (artefatos): O resultado da atividade
 - Papeis: Responsabilidades das pessoas envolvidas
 - Pré e Pós Condições: Devem ser verdade antes e depois da realização da atividade

Tipos de Processos

.Baseado em Planos

–todas as atividades são planejadas com antecedência, mais fácil de medir o progresso

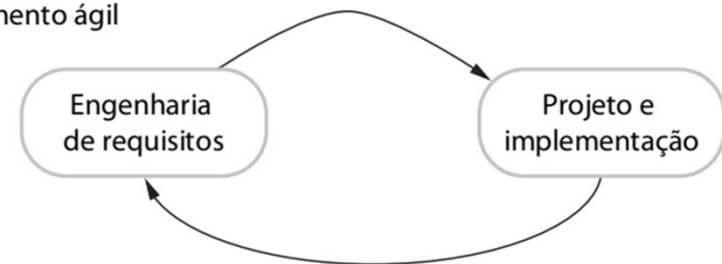
.Ágeis

–planejamento incremental, mais fácil de realizar alterações

Desenvolvimento baseado em planos



Desenvolvimento ágil



Tipos de Processos

.A maior parte dos processos práticos é um misto dos dois, apesar de que as ágeis são cada vez mais frequentes

NÃO EXISTE PROCESSO CERTO OU ERRADO!

Modelos de Processo

.Modelo em Cascata

- Baseado em Planos. Fases distintas e separadas de especificação e desenvolvimento

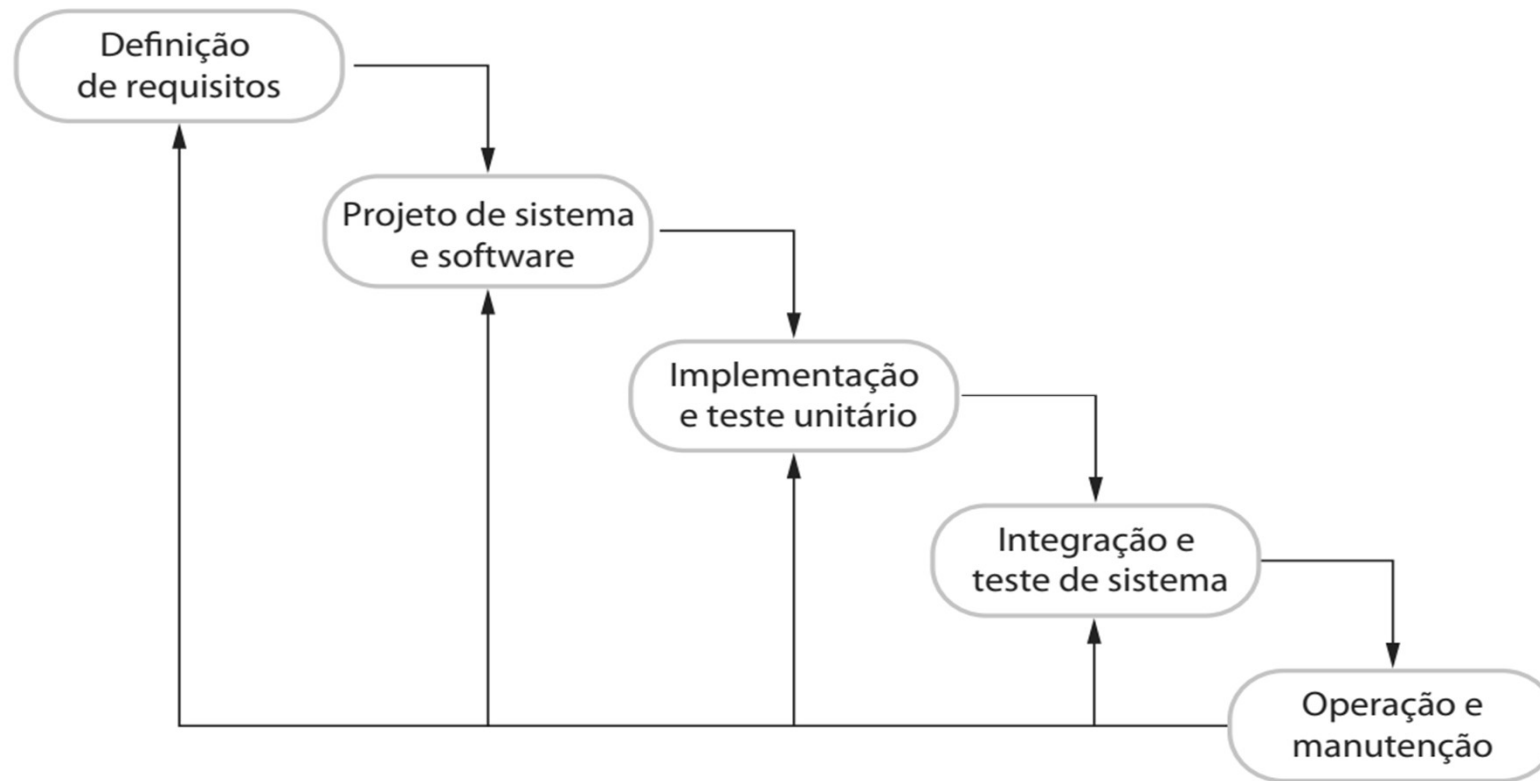
.Desenvolvimento Incremental

- Baseado em Planos ou Ágil. Especificação, desenvolvimento e validação são intercalados

.Integração e Configuração (Reúso)

- Baseado em Planos ou Ágil. É montado a partir de componentes existentes e configuráveis

Modelo em Cascata



Análise e Definição de Requisitos

- Identificação de necessidades e coleta inicial de dados
- Estudo de viabilidade (técnica e econômica)
- Especificação e validação dos requisitos junto ao cliente
- O documento final diz “o que” será o sistema

Projeto de Sistema e Software

.Diz “como” o sistema será desenvolvido

–Hardware

–Interface

–Modelo de Dados

–Componentes

–Algoritmos

Implementação e Teste Unitário

- Traz as partes do sistema à vida
- Implementa e testa de forma independente os elementos de software identificados no projeto
- Garante que cada unidade de software funciona corretamente

Integração e Teste de Sistema

- As unidades individuais são integradas com o hardware para formar o sistema final
- Com o sistema completo, é testado para garantir que os requisitos foram atendidos
- Após esta etapa é entregue ao cliente

Operação e Manutenção

- O sistema é instalado e colocado em uso
- A manutenção pode envolver
 - Treinamento
 - Correção de erros
 - Melhorias na implementação de unidades
 - Modificação para incluir novos requisitos

Cascata: Aplicação do Modelo

- .Cada estágio produz um documento que deve ser aprovado pela equipe e pelos clientes
- .Cada etapa só inicia após a finalização da anterior
- .Na prática os documentos podem ser modificados para refletir alterações de fases seguintes (e devem ser aprovados novamente)

Exercício: Definição

Um cientista coleta uma série de dados no formato de pares (X, Y) . Ele gostaria de um software que:

- Calcule a média dos dados
- Calcule o desvio padrão dos dados
- Calcule a reta que melhor se ajusta aos dados
- Exiba graficamente os pontos e a reta

Exercício: Baseado no que você aprendeu

.Análise e Definição de Requisitos

–Está suficientemente especificado? Precisa saber mais alguma coisa?

.Projeto de Sistema e Software

–Qual tipo de máquina vai usar? Qual linguagem? Como será a interação do usuário com o programa? Quais funções serão necessárias? Quais as entradas e saídas dessas funções? Como calcular o que ele quer?

Exercício: Baseado no que você aprendeu

•Implementação e Teste Unitário

- Tente implementar independentemente cada função que planejamos (pode ser feito separadamente por grupos)
- Teste as funções para verificar se funcionam corretamente

•Integração e Teste de Sistema

- Junte todas as funções em um programa único
- Verifique se ainda funciona

Exercício: Baseado no que você aprendeu

- O cliente gostaria de adicionar um botão para apagar o gráfico
- O que precisa ser feito?

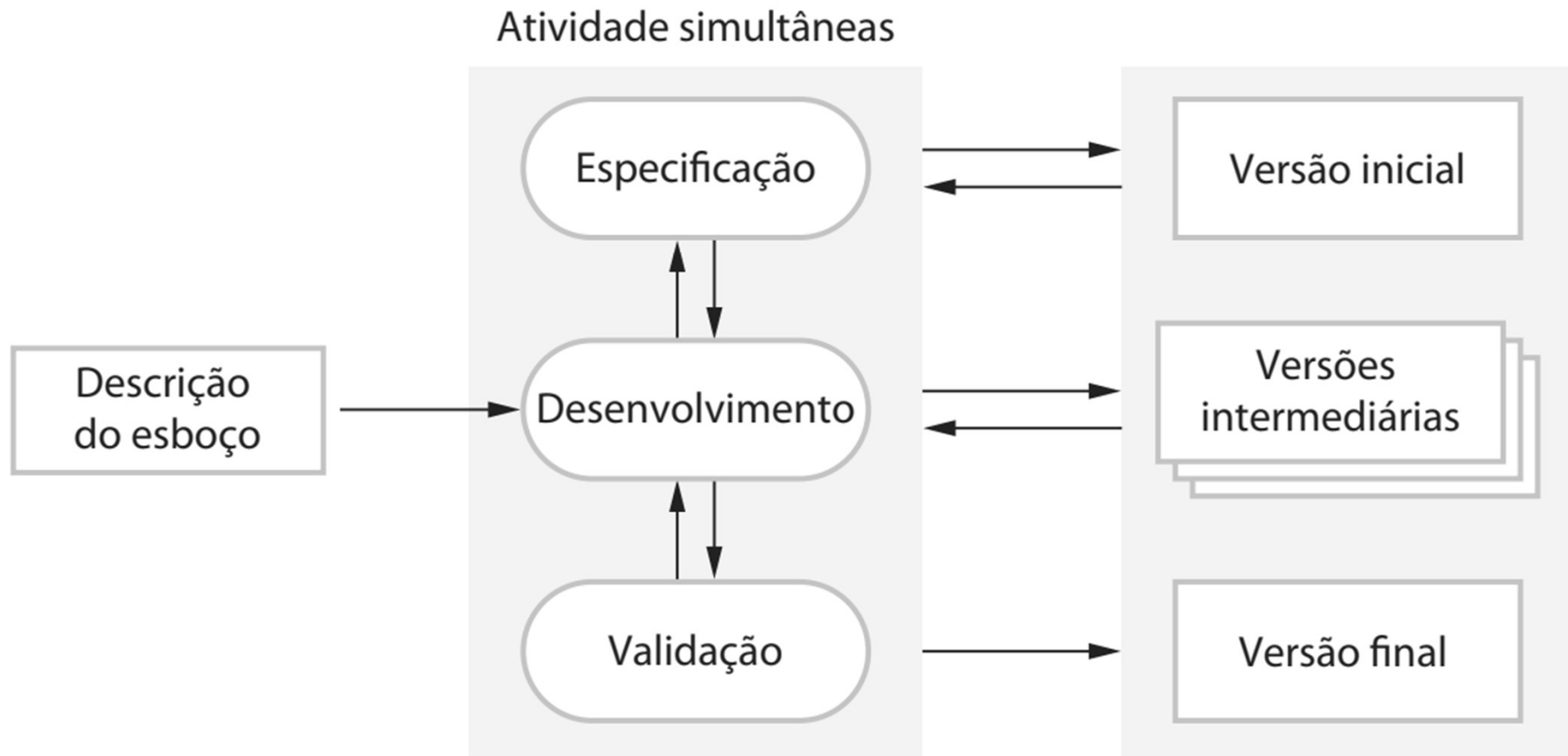
Cascata: Vantagens

- .A documentação é produzida a cada fase
- .Processo visível
- .Fácil de monitorar o progresso
 - Comparar com o plano

Cascata: Desvantagens

- Pouco flexível, dificulta alterações
- Pode bloquear parte da equipe
- O cliente tem que ser paciente
 - Só tem um software no final do processo

Desenvolvimento Incremental



Incremental: Vantagens

- Mais fácil de acomodar mudanças
- Mais fácil de obter o retorno do cliente
- Entrega mais rápida de um software útil ao cliente

Incremental: Desvantagens

- O processo não é visível

- Gerentes precisam de “entregáveis” (documento ou protótipo) regulares para medir o progresso.

- Se o desenvolvimento é rápido, não é necessário documentar todas as versões do sistema

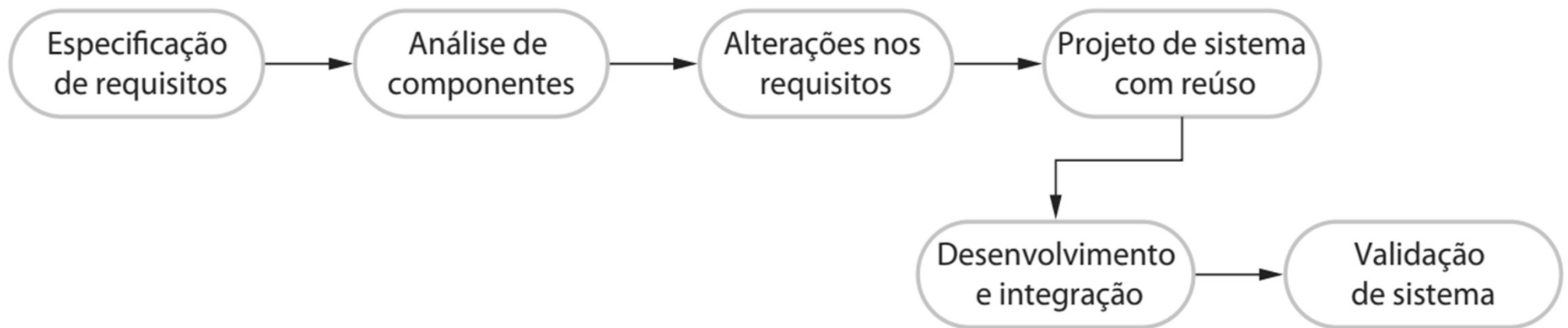
- A estrutura geral tende a degradar com a inclusão de novos elementos

Exercício de Imaginação

• Para o sistema anterior criamos os seguintes protótipos:

- Faz apenas o cálculo e exibição da média
 - Acrescenta o cálculo e exibição do desvio padrão
 - Acrescenta a exibição dos pontos
 - Acrescenta o botão de apagar
 - Acrescenta a exibição da reta
 - Modifica o local de exibição da média e desvio padrão
- Será que o resultado é igual? O que você acha? Discuta

Integração e Configuração (Reúso)



Integração e Configuração (Reúso)

- Integra softwares já prontos para criar um novo sistema
- Os elementos reutilizado podem ser configurados para se adaptar aos requisitos
- Em muitos negócios esta tem sido a abordagem padrão

Tipos de Componentes de Reutilizáveis

- .Sistemas stand-alone configuráveis**
- .Coleções de objetos ou funções desenvolvidas como pacotes ou bibliotecas**
- .Web services disponíveis para chamada remota**

Reuso: Vantagens

.Riscos reduzidos

.Custos reduzidos

.Entrega mais rápida

Reuso: Desvantagens

- Sistema pode não atender os requisitos do cliente por limitações dos componentes disponíveis
- Não temos controle sobre a evolução dos elementos reutilizados

Desenvolvimento Ágil

Como surgiu?

- A abordagem baseada em planos gera uma grande sobrecarga (overhead) na elaboração de uma documentação detalhadas
 - Requisitos, Arquitetura do Sistema, Arquitetura do Software, Modelo de Dados...
- Insatisfeitos, um grupo propõe os métodos ágeis que focam no programa e não no seu projeto e documentação

Princípios dos Métodos Ágeis

- .Envolvimento do cliente**
- .Entrega incremental**
- .Pessoas, não processos**
- .Aceitar as mudanças**
- .Manter a simplicidade**

Extreme Programming (XP)

- .Na XP os requisitos são expressos como cenários (estórias do usuário) que são quebrados em tarefas
- .Os programadores trabalham em pares e desenvolvem testes para as tarefas antes de escreverem o código
- .Quando o novo código é integrado, todos os testes devem ser executados com sucesso
- .Há um curto intervalo entre as entregas (~15 dias)

Cenários como Requisitos

- .O cliente é parte do time de desenvolvimento e expressa os requerimentos como histórias**
- .O time de desenvolvimento quebra as histórias em tarefas. Elas são a base do planejamento e estimativa de custos.**
- .O cliente escolhe a história que será incluída na próxima entrega baseado nas suas prioridades e no tempo disponível.**

Cenário: Prescrição de Medicamento

- O registro do paciente deve ser aberto para entrada. Clique no campo medicação e selecione “atual”, “nova ” ou “formulário”
- Se selecionar “atual”, é pedido para verificar a dose; se quiser mudar a dose, coloque o novo valor e confirme.
- Se selecionar “nova”, escreva as primeiras letras do nome do medicamento. Você verá uma lista com os medicamentos que iniciam com estas letras. Escolha o medicamento. Confirme o medicamento. Insira a dose e confirme.
- Em todos os casos o sistema verifica se a dosagem está dentro dos limites permitidos. Se não estiver, solicita alteração.

Tarefa: Verificar Dosagem

- . A verificação de dosagem é uma medida de segurança para evitar doses perigosamente altas ou baixas.
- .Usando o id do medicamento, pesquise no formulário as doses mínima e máxima.
- .Se estiver fora da faixa, exiba uma mensagem de erro informando que a dose é muito alta ou baixa, a depender da situação, e solicitando a alteração. Se está na faixa, ative o botão “Confirmar”.

Exercício

.Quais outras tarefas você consegue identificar nesse cenário?

Refatoração

- A engenharia de software tradicional diz que vale a pena gastar tempo e esforço antecipando mudanças pois reduz custos em etapas futuras
- XP diz que não, pois não dá para prever de forma confiável quais mudanças ocorrerão. Só há alteração quando a mudança surgir.
- Mas mudanças incrementais podem levar a uma degradação da estrutura do sistema

Refatoração

- XP propõe melhoramento constante do código para facilitar mudanças quando for necessário
- Melhora a compreensão do código e reduz a necessidade de documentação
- Como o código está limpo e bem estruturado é fácil de alterar
 - Desde que não precise mudar a arquitetura

Exemplo de Refatoração

- Reorganização de classes para remover código duplicado
- Organizar e renomear métodos e variáveis para facilitar a compreensão
- Trocar códigos em linha por chamadas a métodos

Desenvolvimento orientado a Testes

- Desenvolver o teste primeiro

- Cliente ajuda no projeto dos testes

- Testes automatizados permitem a realização de todos os testes a cada nova entrega

Problemas com os Testes

- Muitas vezes são incompletos
- Em certas situações são muito difíceis de escrever

Teste: Verificar Dosagem

.Entrada:

- Um número representado uma dose em mg
- Um número representado o número de doses por dia

.Testes:

- Entradas em que uma dose é muito alta ou muito baixa
- Entradas em que o produto dose x frequência é muito alto ou muito baixo
- Entradas em que o produto dose x frequência está na faixa permitida

.Saída:

Programação em Pares

- Como o nome diz, trata-se de programadores trabalhando em duplas
- As duplas são dinâmicas, mudando periodicamente
- Serve como um sistema de revisão
- Encoraja o refatoramento e a escrita de código legível
- Compartilha o conhecimento do código, evitando problemas quando um membro precisa sair da equipe

Gerencia de Projetos Ágeis: Scrum

- O Scrum é um *framework* para a organização e planejamento de projetos ágeis
- Há três fases no Scrum
 - Início, define em linhas gerais o objetivo do projeto e a arquitetura de software
 - Série de ciclos de *sprint*, cada ciclo desenvolve um incremento
 - Fechamento, encerra o projeto e completa a documentação necessária, como ajudas e manuais

Terminologia

.Time de Desenvolvimento

–Um grupo auto-organizado de programadores, não mais de 7 pessoas. Responsáveis por desenvolver o programa e documentos

.Produto Potencialmente Entregável

–O incremento entregue em um sprint. Potencialmente entregável significa que está pronto, não é necessário mais trabalho para incorporar no produto final. Na prática, nem sempre é obtido.

Terminologia

.Backlog do Produto

–Lista de coisas a fazer que o time Scrum deve cumprir. Podem ser definições de funcionalidades (features) do software, requisitos, cenários ou tarefas extras necessárias como definição de arquitetura e documentação

.Dono do Produto (Product Owner)

–Um indivíduo (ou grupo) cujo papel é identificar e priorizar as funcionalidades e requisitos, além de verificar os backlogs para garantir que o projeto atende as necessidades. Em geral é o cliente.

Terminologia

.Daily Scrum

–Um encontro diário que verifica o progresso e define os trabalhos a serem feitos no dia. Idealmente é curto e com todo o time

.Scrum Master

–Responsável por garantir que o processo Scrum é seguido e guia o time.
Responsável por fazer a interface com o resto da empresa

Terminologia

.Sprint

–Uma iteração de desenvolvimento. Dura de 2 a 4 semanas.

.Velocidade

–Uma estimativa de quanto Backlog o time consegue cobrir em um sprint. Entender a velocidade do time ajuda a estimar o que pode ser coberto em um sprint e dá uma base para medir o desempenho.

O ciclo de Sprint

•É uma unidade de planejamento.

–Revisa o backlog do produto e identifica prioridades e riscos.

–A seleção envolve todo o time para definir os recursos e funcionalidades a serem desenvolvidos

–Com todos de acordo, a equipe se organiza para desenvolver. Nesta etapa ocorrem os Scrums e a equipe de desenvolvimento está isolada do cliente e do resto da empresa, toda comunicação ocorre através do Scrum Master, que protege a equipe de distrações externas

–No fim, o trabalho é revisado e apresentado

Tudo muito legal mas...

- Como já disse antes, nenhum processo é certo ou errado
- Na prática, diversos fatores externos acabam decidindo o processo mais adequado
- E quase sempre terminamos com um processo misto

Fatores do Sistema

.Qual o tamanho do sistema?

–Métodos ágeis não funcionam muito bem em sistemas grandes

.Qual o tipo do sistema?

–Sistemas que necessitam de muita análise antes de implementar precisam de um projeto detalhado

.Qual a expectativa de vida do sistema?

–Sistemas de longa duração precisam de documentação para as equipes de suporte futuras

.O sistema está sujeito a regulamentação externa?

Fatores Humanos

- Quão bons são os projetistas e programadores no time?
 - Projetos detalhados são mais fáceis de implementar
- Como o time está organizado?
 - Em times distribuídos documentos de projetos podem ser necessários
- Como é a personalidade dos indivíduos?
 - Alguns membros podem não ter disposição para a interação intensa típica de métodos ágeis

Fatores Organizacionais

- O cliente vai estar disposto e disponível para dar *feedback* o tempo todo?
 - Geralmente os clientes não têm tempo para fazer parte em tempo integral da equipe de desenvolvimento. Além disso, o cliente em geral não é uma única pessoa, e temos apenas um representante ali guiando as decisões, é comum que representantes diferentes priorizem aspectos diferentes.
- O desenvolvimento informal ágil se enquadra na cultura da empresa?
 - Em muitas empresas há uma cultura de documentação extensiva.

**Apesar de não ter nada “certo”
ou “errado”, eu posso contar...**

Grandes Mitos Gerenciais

- .Um bom livro de ES basta para fazer bom software**
- .Se o cronograma está atrasado, basta adicionar mais gente à equipe**
- .Se for terceirizado, meus problemas estão resolvidos**

Grandes Mitos do Cliente

- Basta um ideia geral no início do projeto
- Modificações são fáceis de fazer, porque software é flexível

Grandes Mitos do Desenvolvedor

- O único produto a ser entregue é o código
- ES só atrapalha, muito tempo perdido em planejamento e documentação

Grandes Verdades Sobre o Desenvolvimento

- Não estabeleça prazos audaciosos demais
- Nem toda apresentação será um sucesso
- A estrutura hierárquica tradicional só atrapalha
- Preste atenção nos sinais do mercado
- Escolha atributos importantes para o cliente
- O que serve para um cliente pode não servir para outro
- Busque soluções eficientes

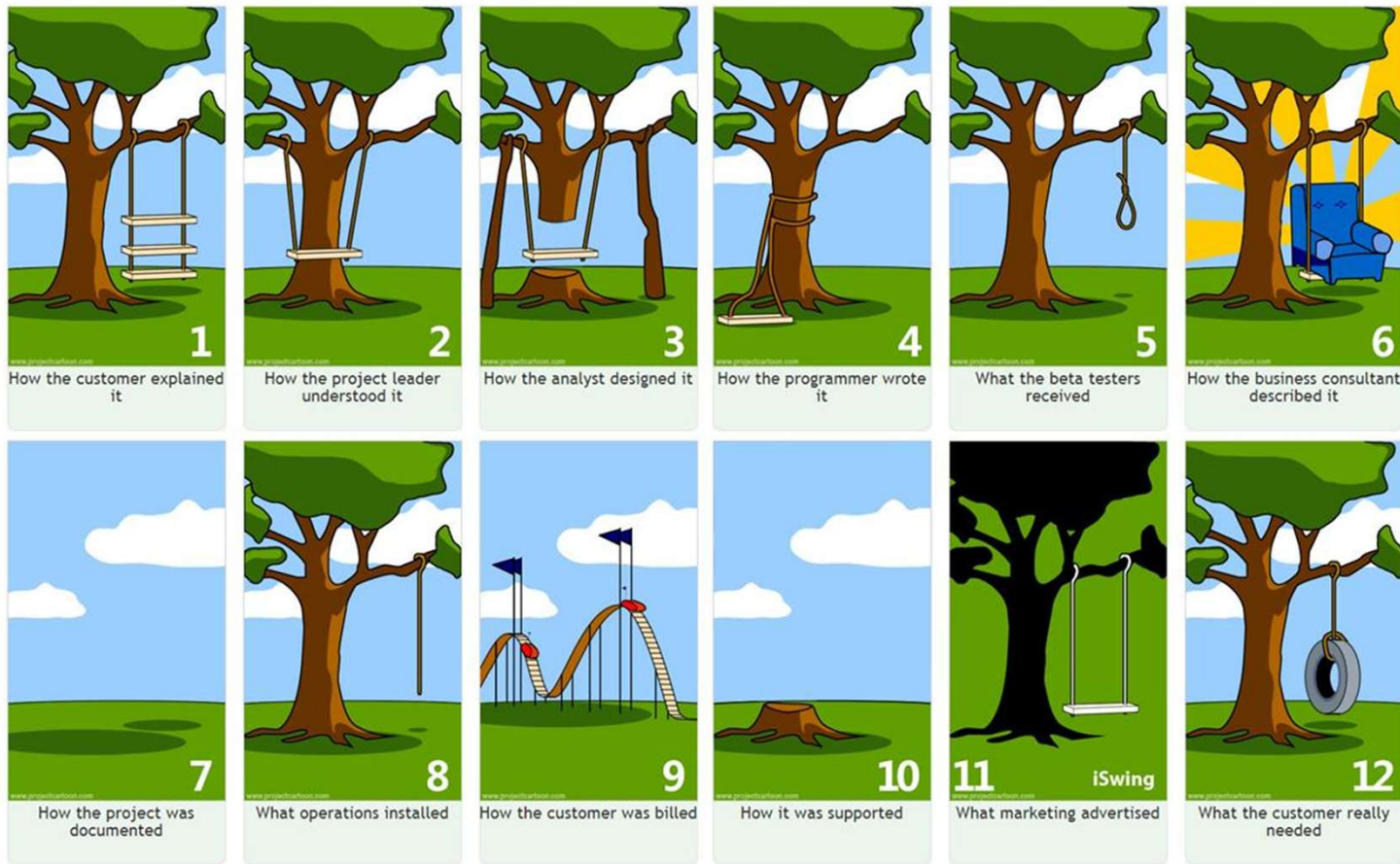
Grandes Verdades Sobre o Desenvolvimento

- Seja inovador
- Escolha a ferramenta certa para cada situação
- Soluções técnicas nem sempre podem ser implantadas
- Experiência em simulações (protótipos) são úteis
- A realização nem sempre sai de acordo com a previsão
- As dificuldades das pessoas devem ser consideradas

Grandes Verdades Sobre o Desenvolvimento

- Acostume-se a trabalhar sob pressão
 - Acredite em você mesmo
 - Ajuda on-line é muito útil...
 - mas não acredite em tudo
-
- Vá até o fim!

Projeto de Software



Princípios de Hooker

- Tem que existir uma razão para fazer o software
 - Se não há razão, melhor não fazer
 - Fazer software consiste em “agregar valor para o usuário”
 - É importante enxergar os reais requisitos
- Keep it Simple, Sir (KISS)
 - “um projeto deve ser o mais simples possível, mas não mais que isso”
 - Fazer de forma simples quase sempre toma mais tempo

Princípios de Hooker

.Mantenha o estilo

–O projeto deve seguir um único estilo

–Padrões e estilos devem ser estabelecidos no início e seguidos por todos

.O que é produzido por você é consumido por outros

–Sempre especifique, projete e codifique pensando que outros vão ler

–Forneça produtos com a mesma qualidade que você exige dos que você consome

Princípios de Hooker

- .Esteja pronto para o futuro
 - Projete pensando na manutenção
- .Planeje para reutilização
 - Pense no problema geral
 - Busque soluções já existentes
- .Pense antes de fazer!
 - avalie alternativas
 - reduza os riscos

Recapitulando

O Supermercado da ES

- .As “prateleiras” da engenharia de software estão cheias de métodos pra produzir software de qualidade
- .Você coloca no seu “carrinho” os processos, métodos e ferramentas que forem adequados à sua situação

Cuidado!

- As ferramentas devem ser adequadas à situação e não adequar a situação às ferramentas
- Menos que o necessário pode levar à desordem
- Mais que o necessário pode emperrar o projeto